Q) What are CASE Tools?

CASE stands for Computer-Aided Software Engineering. CASE tools are software applications that help automate different activities in the software development process. They assist developers in analysis, design, coding, testing, documentation, and maintenance of software systems.

✓ Components of CASE Tools

1. Repository
A central database that stores all project information such as models, diagrams, code, and documentation.

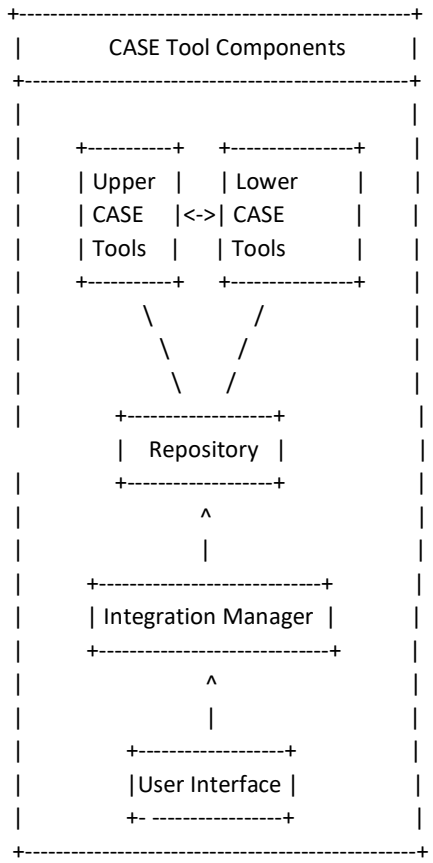
2. User Interface
The part of the tool developers interact with. Provides menus, editors, diagrams, and navigation options.

3. Upper CASE Tools
Used in early stages of development like requirements gathering, analysis, and design.

4. Lower CASE Tools
Used in later stages of development like coding, testing, and maintenance.

5. Integration Manager
Manages the communication and coordination among all other components. Ensures consistency of project information

Diagram of Components:



ADVANTAGES & DISADVANTAGES OF CASE COMPONENTS

BASE	ADVANTAGES	BASE	DISADVANTAGES
1) Increases Productivity	Automates repetitive tasks like code generation, diagrams, and documentation. Saves developers' time.	**1) High Cost**	Purchase, implementation, and maintenance are expensive.
2) Improves Software Quality	Checks models for errors and inconsistencies. Reduces human errors.	**2) Complexity**	Tools are complex and require training to use effectively.
3) Better Project Management	Helps in planning, scheduling, and tracking project progress.	**3) Resistance to Change**	Team members may hesitate to adopt new tools and workflows.
4) Promotes Standardization	Encourages the use of standard methods and notations like UML.	**4) Not Suitable for All Projects**	Small or short-term projects may not benefit enough to justify the cost.
5) Easier Maintenance	Central repository makes updating and modifying software simpler.	**5) Tool Dependency**	Teams can become dependent on specific tools, making migration difficult.

- TECHNICAL ASPECTS & MANAGERIAL ASPECTS

TECHNICAL ASPECTS	MANAGERIAL ASPECTS
1) Automates design and coding (e.g., generating code from models).	1) Supports project planning by helping estimate time, cost, and resources.
2) Ensures consistency across design, code, and documentation.	2) Monitors project progress with dashboards and reports.
3) Facilitates testing through test case generation and defect tracking.	3) Manages resources by tracking workload and assignments.
4) Improves quality by error checking and validation.	4) Enforces quality standards and development methodologies.
5) Produces clear documentation automatically.	5) Improves team communication via shared data and models.

Q) What is Cohesion?

Cohesion is a measure of how closely the elements (functions, data) within a module are related to each other.

High cohesion means all parts of the module work together to perform one well-defined task.

Low cohesion means the module has unrelated or loosely connected tasks, making it harder to understand, maintain, and reuse.

- Goal:-

We always prefer high cohesion because it improves clarity, reliability, and reusability.

- Types of Cohesion (from worst to best):

1. Coincidental Cohesion (Worst)

-The module's elements have no meaningful relationship.

-Just a collection of random functions.

-Example: A module containing functions for file handling, printing, and database connection.

2. Logical Cohesion

-Elements are logically categorized to do similar things, but only one is selected during execution.

-Example: A module with different error-handling routines selected by a control flag.

3. Temporal Cohesion

-Elements are related by the fact that they are executed together at the same time.

-Example: A module that performs initialization tasks when a system starts.

4. Procedural Cohesion

-Elements are related and must be executed in a specific order.

-Example: A module that validates input and then formats it.

5. Communicational Cohesion

-Elements operate on the same input or output data.

-Example: A module that reads data from a file and processes the same data.

6. Sequential Cohesion

-The output from one part is the input to another part.

-Example: A module that takes raw data, sorts it, and then formats it for display.

7. Functional Cohesion (Best)

-All elements contribute to exactly one well-defined task.

-Example: A module that calculates the area of a circle given the radius.

Q) What is Coupling?

Coupling refers to the degree of interdependence between software modules.
It shows how closely connected two modules are.
High coupling means modules are strongly dependent on each other, making changes difficult and error-prone.
Low coupling means modules are independent, making software easier to maintain, test, and reuse.

- Goal:

We aim for low coupling and high cohesion to create well-structured software.

- Types of Coupling

Coupling types are classified from strongest (worst) to weakest (best)

1. **Content Coupling (Worst)**
   -Definition: One module directly accesses or modifies the internal data or logic of another module.
   -Example: Module A changes a local variable of Module B.
   -Problem: Any change in Module B can break Module A.

2. **Common Coupling**
   -Definition: Two or more modules share the same global data.
   -Example: Multiple modules updating the same global variable.
   -Problem: Difficult to track which module changes the data, leading to bugs.

3. **External Coupling**
   -Definition: Modules share externally imposed data formats, protocols, or device interfaces.
   -Example: Modules depending on a shared file format.
   -Problem: Changes in external systems require changes in all related modules.

4. **Control Coupling**
   -Definition: One module controls the behavior of another by passing control information, such as flags or commands.
   -Example: Passing a control flag to select which function to execute.
   -Problem: Reduces modularity because the calling module must know internal details.

5. **Stamp Coupling (Data-structured Coupling)**
   -Definition: Modules share a composite data structure (e.g., a record or object), but only use part of it
   -Example: Passing an entire customer record when only the customer ID is needed.
   -Problem: Unnecessary data sharing increases dependency.

6. **Data Coupling (Best Practicable)**
   -Definition: Modules share only the data required, passed as simple parameters.
   -Example: Passing individual values like integers, strings.
   -Advantage: Low dependency and clear interfaces.

7. **No Coupling (Ideal but Rare)**
   -Definition: Modules are completely independent, with no interaction.
   -Example: Completely separate modules performing unrelated tasks.
   -Note: This is rare in practical systems.

-- Diagram: Coupling Types in Order

Worst Coupling
|
v
Content Coupling
Common Coupling
External Coupling
Control Coupling
Stamp Coupling
Data Coupling
^
|
Best Coupling

-- Importance/ Advantages of Low Coupling:-

1) Easier to maintain and modify modules.
2) Increases reusability of modules in other systems.
3) Reduces the risk of errors when changes are made.
4) Supports parallel development.

-- Problems of High Coupling Between Modules:-

High coupling means modules are tightly connected and strongly dependent on each other. This causes several issues:

1. Difficult Maintenance
   -Changes in one module often force changes in other modules.
   -Even small updates can create a ripple effect throughout the system.

2. **Reduced Reusability**
   -Highly coupled modules are hard to reuse in other projects.
   -They depend on many other parts of the system to work properly.

3. **Complex Testing**
   -Testing becomes complicated because you cannot isolate modules easily.
   -You have to test them together, increasing time and effort.

4. **Low Flexibility**
   -It is difficult to replace or improve a module without impacting others.
   -This makes adapting to new requirements harder.

5. **Higher Error Propagation**
   -Bugs in one module can spread quickly to dependent modules.
   -Locating and fixing errors takes more time.

6. **Poor Understandability**
   -Code becomes harder to read and understand because of many interconnections.
   -New developers struggle to grasp the system logic.

Q) What is Denormalization?

Denormalization is the process of introducing redundancy into a database table by combining tables or adding duplicated data to improve read performance.

In normalization, data is split into multiple related tables to eliminate redundancy and maintain data integrity.

In denormalization, some of these tables are merged back together, or redundant columns are added to reduce the need for joins, making queries faster.

--When to use it?

-When performance is more important than storage efficiency.
-In read-heavy applications where many joins slow down query speed.

--Example of Denormalization

Let's take a simple normalized example:

-- Normalized Tables:

1. **Employee Table:**

EmpID	EmpName	DeptID
1	Alice	101
2	Bob	102

2. **Department Table:**

DeptID	DeptName
101	HR
102	IT

☞ If you want to get the employee name and their department name, you need a JOIN query.

After Denormalization:

Employee Table with Department Name added:

EmpID	EmpName	DeptID	DeptName
1	Alice	101	HR
2	Bob	102	IT

Q What Changed?

-We duplicated the DeptName column into the Employee table.
-No need to join with the Department table to get department names.
-Query becomes faster but data redundancy is introduced.

-- Advantages of Denormalization:

1) Faster SELECT queries.
2) Fewer joins needed.
3) Better performance for reporting.

-- Disadvantages:

1) Data redundancy.
3) Higher risk of inconsistency (e.g., if DeptName changes, it must be updated in all rows).

-: DIFFERENCE BETWEEN COHESION & COUPLING:-

ASPECT	COHESION	COUPLING
1) Definition	Cohesion refers to the degree to which elements of a module belong together.	Coupling refers to the degree of interdependence between different modules.
2) Focus	Focuses on **how well the internal elements** of a module work together.	Focuses on **how much one module depends** on other modules.
3) Desirable Form	**High cohesion** is desirable.	**Low coupling** is desirable.
4) Effect on Maintainability	High cohesion improves **maintainability and readability**.	High coupling makes maintenance **difficult and error-prone**.
5) Dependency	Elements are **internally dependent** and focused on a single task.	Modules are **externally dependent** on each other.
6) Change Impact	Changes in one part of the module have **minimal effect** on others.	Changes in one module can **impact other coupled modules**.
7) Example	A module performing all file-related operations like open, read, write.	A module calling functions from another module for basic operations.
8) Measurement	Measured to check **module consistency**.	Measured to check **inter-module connectivity**.
9) Goal in Design	To increase cohesion to make modules more **focused and logical**.	To reduce coupling to make modules more **independent**.

BEST

OF

LUCK